

课程笔记

CS146S Week 2: The Anatomy of Coding Agents (MCP)

基于 Mihail Eric 授课内容整理

2025 年 10 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 2 lectures (Mon + Fri)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 从零构建 Coding Agent	2
1.1 什么是 Coding Agent	2
1.2 “秘密武器”：Claude Code 的设计哲学	2
1.3 本章小结	2
2 Model Context Protocol (MCP)	3
2.1 为什么需要 MCP	3
2.2 MCP 是什么	3
2.3 MCP 架构详解	3
2.3.1 核心术语	3
2.3.2 工作流程	3
2.3.3 传输层	4
2.4 MCP 的局限性	4
2.5 本章小结	4
3 从协议到产品：MCP 为什么会成为分水岭	4
3.1 它解决的不只是连接问题，而是生态协作问题	4
3.2 真正困难的部分：协议统一后，设计责任才开始出现	4
3.3 本章小结	5
4 实践视角：一个可维护 Coding Agent 的三层检查表	5
4.1 最容易被忽略的是会话和上下文治理	5
4.2 本章小结	6
4.3 什么时候不该把问题简单归结为“上一个 MCP Server”	6
5 总结与延伸	6
5.1 总结表	6
5.2 关键点	6
5.3 拓展阅读	7

1 从零构建 Coding Agent

1.1 什么是 Coding Agent

Coding Agent 是一种能够自主执行编程任务的 AI 系统。其核心组件包括：

- **System Prompt**: 定义 Agent 的行为和指令
- **User Prompt**: 用户的自定义请求
- **Assistant Prompt**: LLM 的响应
- **Tool Set**: Agent 可调用的工具集合

Coding Agent 的核心循环

一个 Coding Agent 的基本工作流程极其简单：

1. 从终端读取用户输入，持续追加到对话历史
2. 告知 LLM 可用的工具（Read_file、List_dir、Edit_file 等）
3. LLM 在适当时机请求调用工具
4. 离线执行工具调用并返回结果
5. 重复上述过程直到任务完成

1.2 “秘密武器”：Claude Code 的设计哲学

深入分析 Claude Code 的底层设计，可以发现几个关键的工程决策：

1. **前置 context 注入**：使用小而精准的 prompt 进行 context 前置加载
2. **System reminders 无处不在**：在 system prompt、user prompt、tool calls、tool results 中都插入 <system-reminder> 标签，防止模型在长对话中偏离目标
3. **Command prefix extraction**：从用户输入中提取命令前缀，实现快捷操作
4. **Subagent 派生**：生成子 Agent 处理特定子任务，避免单个 context 过载

Context Drift 问题

在长对话中，LLM 容易“忘记”早期的指令和约束——这被称为 context drift。Claude Code 通过在对话的各个环节反复插入 system reminders 来对抗这一问题。这是一个经过工程验证的关键技巧。

1.3 本章小结

构建一个基础 Coding Agent 的技术门槛其实很低——核心就是 LLM + tool calling 的循环。但要构建一个可靠的 Coding Agent，需要大量的工程细节：context 管理、drift 防护、子任务分解等。

2 Model Context Protocol (MCP)

2.1 为什么需要 MCP

LLM 拥有广泛但**静态**的世界知识——只有在重训时才会更新。要构建真正自主的系统，需要可靠的方式向 LLM 注入动态数据（天气、股价、最新 API 文档等）。

RAG 和 tool calling 是目前最好的答案。但问题在于：

M × N 连接器问题

假设有 M 个 LLM 应用和 N 个第三方工具/API。在 MCP 之前，每个应用需要为每个工具编写独立的集成代码——包括认证、错误处理、速率限制等。这导致了 $M \times N$ 个连接器的维护负担。

2.2 MCP 是什么

MCP (Model Context Protocol) 是 Anthropic 于 2024 年 11 月推出的开放协议，为 LLM 提供了一种**标准化**的工具暴露格式。

MCP 的核心价值

MCP 将工具集成从 $M \times N$ 降低到 $M + N$ ：

- 不需要为每个工具重新实现认证、错误处理、速率限制
- 使用 JSON-RPC 强制执行一致的输出格式
- 从 Language Server Protocol (LSP) 扩展而来，但支持**主动的** (proactive) agentic workflow，而非 LSP 的纯反应式模式

2.3 MCP 架构详解

2.3.1 核心术语

- **Host**：宿主应用（如 Cursor、Claude Desktop）
- **MCP Client**：嵌入在 Host 中的库（为每个 Server 维护有状态的 session）
- **MCP Server**：轻量级的工具包装器 (wrapper)
- **Tool**：可调用的函数（数据源、API 等）

2.3.2 工作流程

1. Client 调用 `tools/list` 向 MCP Server 查询：“你能做什么？”
2. Server 返回 JSON 描述每个工具（名称、摘要、JSON Schema）
3. Host 将这些 JSON 注入模型的 context
4. 用户 prompt 触发模型，模型输出结构化的 tool call
5. MCP Server 执行调用，对话继续

2.3.3 传输层

MCP 提供两种传输机制：

- **stdio**：标准输入/输出，适用于本地进程通信
- **SSE** (Server-Sent Events)：适用于远程服务通信

2.4 MCP 的局限性

当前 Agent 处理多工具的能力有限

- Agent 在面对大量工具时表现不佳——工具描述会快速消耗 context window
- API 设计需要面向 AI 原生 (AI-native)，而非沿用传统的刚性 API 设计
- 复杂的 API 参数和嵌套结构会显著降低工具调用的准确率

2.5 本章小结

MCP 是 AI 编程工具生态的重要基础设施，通过标准化的协议解决了 LLM 与外部工具集成的 $M \times N$ 问题。理解 MCP 的架构 (Host → Client → Server → Tool) 和通信流程是构建和使用 AI 编程工具的关键。

3 从协议到产品：MCP 为什么会成为分水岭

3.1 它解决的不只是连接问题，而是生态协作问题

Week 2 对 MCP 的介绍，很容易被误解为“又一个 tool calling 协议”。但课程想强调的更深一层含义是：**MCP 真正统一的是工具生态的协作边界，而不只是调用语法**。在没有 MCP 的世界里，每个 Agent 产品都要自己决定认证方式、工具描述格式、错误返回结构和会话状态管理；这使得工具作者和 Agent 产品方都不断重复造轮子。

MCP 之所以重要，是因为它把工具从项目私有实现变成了生态公共接口

- 对工具提供方来说，写一次 MCP Server，就能被多个 Host/IDE/Agent 复用；
- 对 Agent 产品来说，不必为每个工具单独适配 schema、错误格式与权限模型；
- 对用户来说，工具切换的成本显著下降，工作流可以跨产品迁移。

这和历史上的 LSP 很像。LSP 并没有发明“补全”或“跳转定义”，但它统一了编辑器与语言服务之间的接口，于是整个生态的创新速度被抬高。MCP 对 Agent 工具生态扮演的是类似角色，只不过作用对象从“编辑器能力”变成了“模型可调用的外部能力”。

3.2 真正困难的部分：协议统一后，设计责任才开始出现

协议统一并不意味着 Agent 就会自然用好工具。相反，接口一旦标准化，大家更容易看到另一个问题：**很多现有 API 根本不是为模型使用而设计的**。

AI-native API 设计不能简单照搬给人类开发者的 API

- 参数过多、嵌套太深、状态切换太隐式，会显著降低模型调用成功率；
- 返回结果如果缺少简洁摘要和明确错误原因，模型很难做后续决策；
- 工具如果不暴露权限边界和副作用说明，Agent 很容易做出高风险调用。

设计维度	传统 API 常见做法	AI-native 工具更优做法
参数设计	参数多、结构深、默认值隐式	参数尽量扁平，必要信息显式给出
错误处理	面向工程师的长错误栈	提供可供模型恢复的结构化错误信息
副作用说明	依赖文档阅读理解	在工具 schema 中直接说明风险与影响范围
返回结果	原始 JSON/对象堆叠	同时提供摘要字段和机器可解析字段

表 1: MCP 解决了协议统一问题，但工具本身是否 AI-native 仍决定了可用性上限

3.3 本章小结

MCP 的意义不在于又多了一个协议，而在于它为 Agent 工具生态提供了公共接口。真正的下一步挑战，是让工具本身也从“给人类开发者看的 API”演化为“给模型稳定调用的 AI-native API”。

4 实践视角：一个可维护 Coding Agent 的三层检查表

4.1 最容易被忽略的是会话和上下文治理

Week 2 前半部分用最小 Coding Agent 展示了 tool loop 的核心结构，后半部分再引入 MCP。把两部分放在一起看，会发现一个更实际的工程问题：一个能长期稳定工作的 Coding Agent，关键不只在会调用工具，而在于怎么管理上下文、状态和失败恢复。

三层检查表

1. **模型层**：系统提示词是否清楚说明任务、边界和工具使用原则；
2. **工具层**：工具 schema 是否稳定、返回结果是否适合下一步推理；
3. **会话层**：上下文是否会膨胀、是否有 reminder、失败后如何恢复。

这也是为什么课程里会强调 context 前置加载、system reminders 和 subagent 分流。这些看似“产品技巧”的东西，实际上是在弥补当前模型在长期任务上的脆弱性。Agent 不是只要会调用工具就够了，它还必须能在长时间、多步骤、多文件的工作中保持目标一致性。

4.2 本章小结

如果把 Week 2 学到的内容真正带回工程实践，一个可靠的 Coding Agent 至少要同时过三道关：模型是否知道该做什么、工具是否适合模型调用、会话是否有足够的上下文治理能力。这三者缺一不可。

4.3 什么时候不该把问题简单归结为“上一个 MCP Server”

Week 2 还有一个很值得保留的工程判断：不是所有集成问题都应该第一时间被包装成一个新的 MCP Server。协议标准化会降低接入成本，但它不会自动解决权限设计、任务切分和观测性问题。

MCP 不是系统设计的替代品

- 如果底层 API 本身不稳定，MCP 只会把不稳定更标准地暴露出来；
- 如果任务边界不清晰，模型即便能调用工具，也可能在错误的层面上自动化；
- 如果缺少日志、重试和回滚机制，协议统一反而可能放大错误传播范围。

这也是为什么课程一边讲协议，一边仍然强调 agent loop、context management 与产品约束。真正可靠的 Coding Agent，从来不是“工具越多越好”，而是“能在明确边界内稳定完成任务”。

5 总结与延伸

Week 2 深入了 Coding Agent 的技术内核。从手动构建一个最小化的 Coding Agent，到理解 MCP 协议如何将工具集成标准化，课程展示了 AI 编程工具“魔法”背后的工程实现。

5.1 总结表

主题	Week 2 的核心结论	工程含义
最小 Agent Loop	LLM + tool call + observation 已能形成基本闭环	原理简单，但上下文与错误处理决定可用性
Claude Code 经验	reminder、context 预载、subagent 分流都很关键	产品设计直接影响模型稳定性
MCP 协议	工具集成从 $M \times N$ 下降到 $M + N$	生态创新可以从私有集成走向公共接口
AI-native Tools	协议统一后，工具设计本身成为瓶颈	API 需要更扁平、可恢复、可解释

表 2: Week 2 更像一节 Coding Agent 基础设施设计课，而不只是工具使用课

5.2 关键点

- Coding Agent = LLM + Tool Calling 循环，核心简单但工程细节决定质量
- Claude Code 的关键设计：context 前置加载、system reminders 防 drift、subagent 分流

- MCP 将 $M \times N$ 集成降为 $M + N$ ，使用 JSON-RPC 标准化工具描述
- MCP 架构：Host $\rightarrow$ Client $\rightarrow$ Server $\rightarrow$ Tool
- 当前局限：Agent 处理多工具能力有限，API 需要 AI-native 设计

5.3 拓展阅读

- MCP 官方文档: <https://modelcontextprotocol.io>
- Anthropic MCP 发布博客: <https://www.anthropic.com/news/model-context-protocol>
- JSON-RPC 2.0 规范: <https://www.jsonrpc.org/specification>
- Language Server Protocol: <https://microsoft.github.io/language-server-protocol/>